DATE : / / SUB :

معماری کامپیوتر

Numbering system

سطح منطقی

0 یا 1

نمایش ممیز ثابت $N=(a_{n-1}\cdots a_0 . a_{-1}\cdots a_{-m})_r$

a_{n-1}: MSD — a_{-m}: LSD

بخش صحیح Entegral Part — بخش کسری fractional Part

دو تا شرط داریم: ① $r \geq 2$ ② $0 \leq a_i \leq r-1$

$$N_r = |N| = \sum_{i=-m}^{n-1} a_i \cdot r^i$$

$m=0$: عدد کاملاً صحیح / $n=0$: کاملاً کسری

$r=2$: $a_i \in \{0,1\}$ Binary Digit (Bit)

$r=16$: $0 \leq a_i \leq 15$

سوال ← $N=(213)_{16}$ این عدد اگه در مبنا 16 باشه به صورت $\underline{2}\ \underline{13}$ است یا $\underline{2}\ \underline{1}\ \underline{3}$ است؟

به صورت $\underline{2}\ \underline{1}\ \underline{3}$ است برای رفع این مشکل که به 2 صورت نخونیم از حروف A تا F استفاده می‌کنیم

تبدیل مبناها ← ① تبدیل از مبنای r به مبنای 10

$x=(1011.001)_2 \rightarrow (11.125)_{10}$

② تبدیل از مبنای 10 به مبنای r: بخش صحیح: تقسیم متوالی / بخش کسری: ضرب متوالی

$N=(14.25)_{10} \longrightarrow (1110.01)_2$

$14 \xrightarrow{0} 7 \xrightarrow{1} 3 \xrightarrow{1} 1 \xrightarrow{1} 0$ $\quad 0.25 \xrightarrow{0} 0.5 \xrightarrow{1} 1.0$

③ تبدیل از مبنای r_1 به مبنای r_2 ← یکی از آسانترین راه‌ها اینکه ابتدا به مبنا 10 تبدیل کنیم

$N=(a_{n-1}a_{n-2}\cdots a_0)_r$ $\quad 0 \leq N \leq r^n-1$ ← اعداد بدون علامت

کوچکترین: 0

بزرگترین: $(r-1)(r-1)\cdots(r-1) = r^n-1$

محاسبات: $A+B$ ✓ $\quad A-B=A+(-B)$

سوال: چگونه تعیین کنیم عدد n_1 رقمی در مبنای r_1 در مبنای r_2 به چه مقدار بیت نیاز داریم؟

مثال → $(397)_{10} \rightarrow (\quad)_2$

$256=2^8 \leq 397 < 512=2^9$

جواب → $r_1^{n_1}-1 \leq r_2^{n}-1 \rightarrow n_1 \log r_1 \leq n \log r_2 \rightarrow n=\left\lceil n_1 \frac{\log r_1}{\log r_2} \right\rceil$

نمایش اعداد علامت دار

① Sign-Magnitude $\quad N=(S\ a_{n-2}\cdots a_0)_r$

$S=0$ → عدد مثبت

$S=r-1$ → عدد منفی

مثال: $r=10$: $0123 \rightarrow +123$ $\quad 9123 \rightarrow -123$ $\quad$ 4123 در مبنا 10 نامعتبر

$r=2$: $0101 \rightarrow +5$ $\quad 1101 \rightarrow -5$

SOBHAN

DATE : / / SUB :

محدوده‌ی نمایش : $-(r^{n-1}-1) \le n \le r^{n-1}-1$

محاسبات : 15171-21 ، Φ2

+5 ⑨ 101	-3 ① 011	+3 011
-2 110	-2 110	+2 010
+3 ⑩ 011	-5 101	+5 101

ایراد‌ها: ① دو نمایش مختلف برای 0+ و 0- ② نیاز به عمل محاسباتی تفریق

② روش مکمل ۱

N عدد مثبت n رقمی است.

$(r-1)(r-1)\cdots(r-1)$

$0 \quad 0 \quad \overbrace{\quad\quad}^{n-2} \quad 0$

$-N = \underbrace{r^n - 1}_{r-1 \text{ یک عدد } n \text{ رقمی با ارقام}} - N$

قاعده: تمام ارقام باید از $r-1$ کم شوند.

مثال $N=(0247)_{10} \rightarrow -N=(9752)$

$N=(01011)_2 \rightarrow -N=(10100)_2$

محدوده نمایش : $-(r^{n-1}-1) \le N \le r^{n-1}-1$

روش محاسبات: اعداد رقم به رقم با هم جمع می‌شوند، در صورت ایجاد رقم نقلی، آن را با نتیجه مجدداً جمع کرده.

+5 0101	+3 0011
-2 1101	+2 0010
① 0010	+5 0101
1	
+3 0011	

ایرادها: ① وجود دو نمایش مختلف برای 0+ و 0-

② نیازبه دو عملیات جمع در صورت ایجاد رقم نقلی

از طبقه آخر:

پرسش ① : نشان دهید $-(-N)=N$ ← $-(r^{n-1}-1-N)=r^{n-1}-1-r^{n-1}+1+N=N$

پرسش ② : آیا امکان دارد در جمع $\in AC$ (بیت نقلی طبقه آخر) با نتیجه، مجدداً $\in AC$ ایجاد شود؟ بله، مثال خلاف

③ روش مکمل ② یا مکمل r

N عددی مثبت است.

$-N = r^n - N$

مثال : $r=10 \quad N=01230$

10000	
-01230	
-N = 98770	

اولین رقم غیرصفر از r و بقیه از $r-1$ کم می‌شوند.

محدوده نمایش : $-r^{n-1} \le N \le r^{n-1}-1$

SOBHAN

DATE : / / SUB :

روش محاسبات: اعداد رقم به رقم با هم جمع می‌شوند در صورت ایجاد بیت نقلی از طبقه آخر از آن صرف نظر می‌کنیم.

+3	0011	-3	1101
+2	0010	-2	1110
+5	0101	-5	1011
		+5	0101

$-2^4 + 2^2 + 2^1 + 2^0$

10111 → -9

$$N = (a_{n-1}\, a_{n-2} \cdots a_1 a_0)_2$$

$$|N| = N_r = (-a_{n-1})2^{n-1} + \sum_{i=0}^{n-2} a_i r^i$$

سرریز overflow

+5	0101	-5	1011
+4	0100	-4	1100
+9	1001	-9	0111

در صورتی که یک عملیات محاسباتی نتیجه‌ای خارج از محدوده مجاز نمایش ایجاد کند می‌گوییم سرریز رخ داده است.

c_{n-1}
$a_{n-1} \cdots a_0$ +
$b_{n-1} \cdots b_0$
$c_n \quad s_{n-1} \cdots s_0$

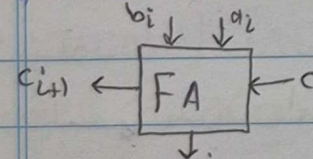
$$OV = \overline{a_{n-1}}\,\overline{b_{n-1}}\,s_{n-1} + a_{n-1}b_{n-1}\overline{s_{n-1}}$$
$$= c_{n-1} \oplus c_n$$

جمع دو عبارت 4 بیتی را در نظر بگیرید:

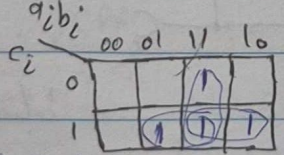
Carry Propagation Adder (CPA)

	a_3	a_2	a_1	a_0
+	b_3	b_2	b_1	b_0
c_4	s_3	s_2	s_1	s_0

یک راه برای پیاده سازی این جمع کننده این است که با توجه به جدول درستی که میکشیم بیاییم گارنومب هاهم بکشیم ولی منطقاً این از ۰ که 256 حالت روتوی کارنومب نشان بدهیم

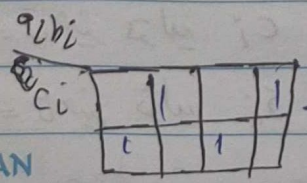


a_i	b_i	c_i	c_{i+1}	s_i
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1



$$c_{i+1} = a_i b_i + a_i c_i + b_i c_i = a_i b_i + c_i(a_i + b_i)$$

Majority func تابع اکثریت



$$\rightarrow s_i = a \hat{} b \hat{} c = a\bar{b}\bar{c} + \bar{a}b\bar{c} + \bar{a}\bar{b}c + abc$$

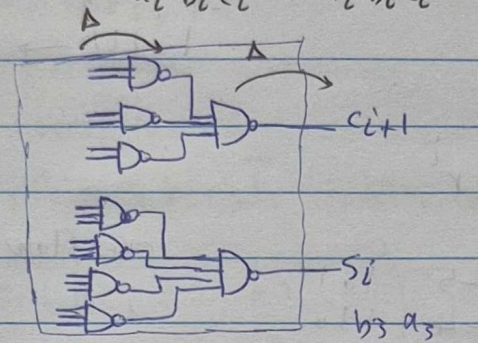
SOBHAN

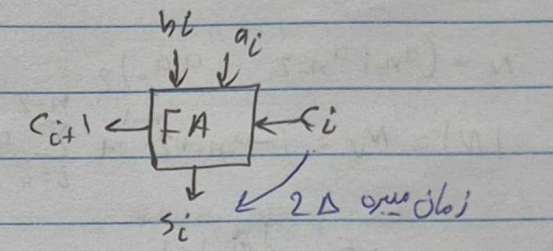
DATE : / / SUB :

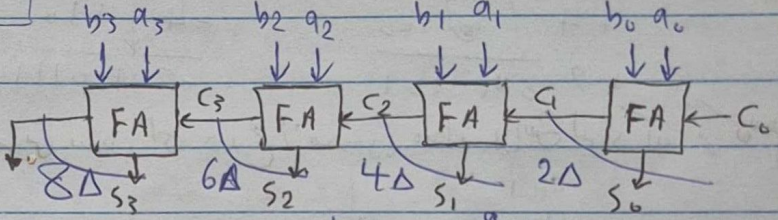
$\overline{c_{i+1}} = \overline{a_i b_i + a_i c_i + b_i c_i} = \overline{a_i b_i} \cdot \overline{a_i c_i} \cdot \overline{b_i c_i} \rightarrow c_{i+1} = \overline{a_i b_i} \cdot \overline{a_i c_i} \cdot \overline{b_i c_i}$

$s_i = \bar{a}_i \bar{b}_i c_i + \bar{a}_i b_i \bar{c}_i + a_i b_i c_i + a_i \bar{b}_i \bar{c}_i$

$\rightarrow s_i = \overline{\bar{s}_i} = \overline{\bar{a}_i \bar{b}_i c_i} \cdot \overline{\bar{a}_i b_i \bar{c}_i} \cdot \overline{a_i b_i c_i} \cdot \overline{a_i \bar{b}_i \bar{c}_i}$





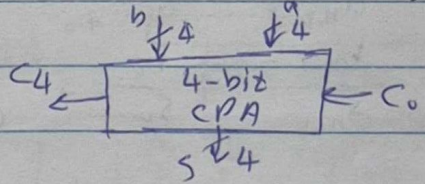


$T_{CPA}(4) = 8\Delta$

Carry Propagation Adder (CPA)

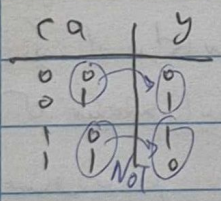
$T_{CPA}(n) = 2\Delta \times n \rightarrow O(n)$

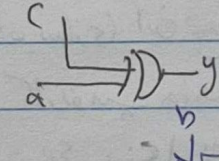
خطی است که خوب نیست.

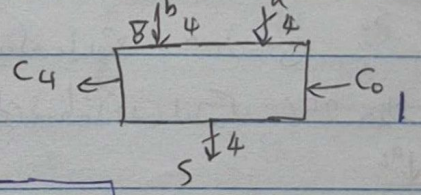


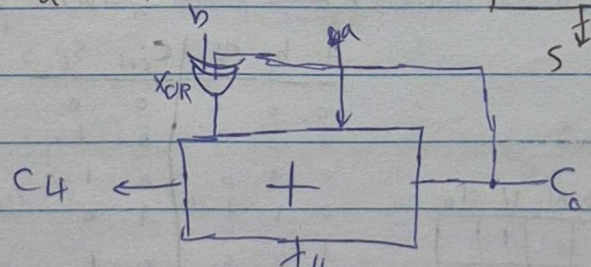
برای اینکه کنترل کنیم این مدار جمع کننده یا تفریق کننده باشد باید از XOR استفاده کنیم

$a - b = a + 2's\ comp(b) = a + 1's\ comp(b) + 1 = a + NOT(b) + 1$

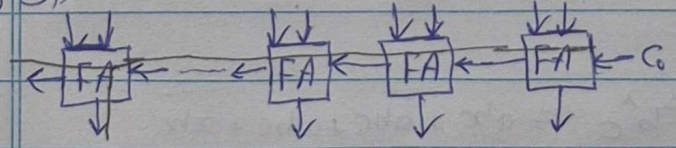








اگر C_0 برابر با 0 باشد مدار جمع می کند اگر C_0 برابر با 1 باشد آنگاه XOR میاد b رو 2'comp میزنه و تفریق کننده صورت می گیرد.



t_s = تاخیر تولید c_i

t_s = تاخیر تولید s_i

$T_{CPA}(n) = (n-1)\, t_c + Max(t_c, t_s)$

c_i تولید میشود و در مرحله آخر هم تغییر $n-1$ بار

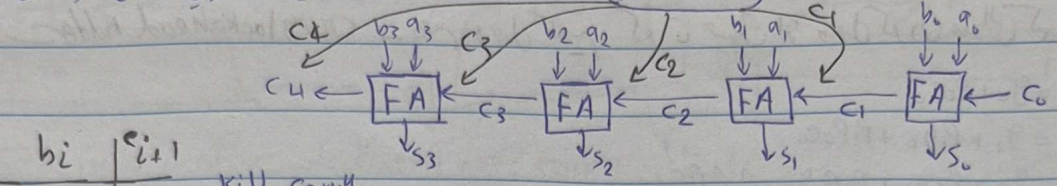
تاخیر برابر با $Max(t_c, t_s)$ میباشد.

SOBHAN

DATE : / / SUB :

ایده کاهش هزینه جمع کننده و طراحی Adder : carrylookahead

پیشبینی کننده carry



a_i	b_i	c_{i+1}	
0	0	0	Kill carry
0	1	c_i	Propagate carry
1	0	c_i	
1	1	1	Generate carry

$$c_{i+1} = \underbrace{a_i b_i}_{g_i} + c_i \underbrace{(\overline{a_i} b_i + a_i \overline{b_i})}_{P_i}$$

$g_i = a_i b_i$, $P_i = \overline{a_i} b_i + a_i \overline{b_i} = a_i \oplus b_i$, $c_{i+1} = g_i + P_i c_i$,

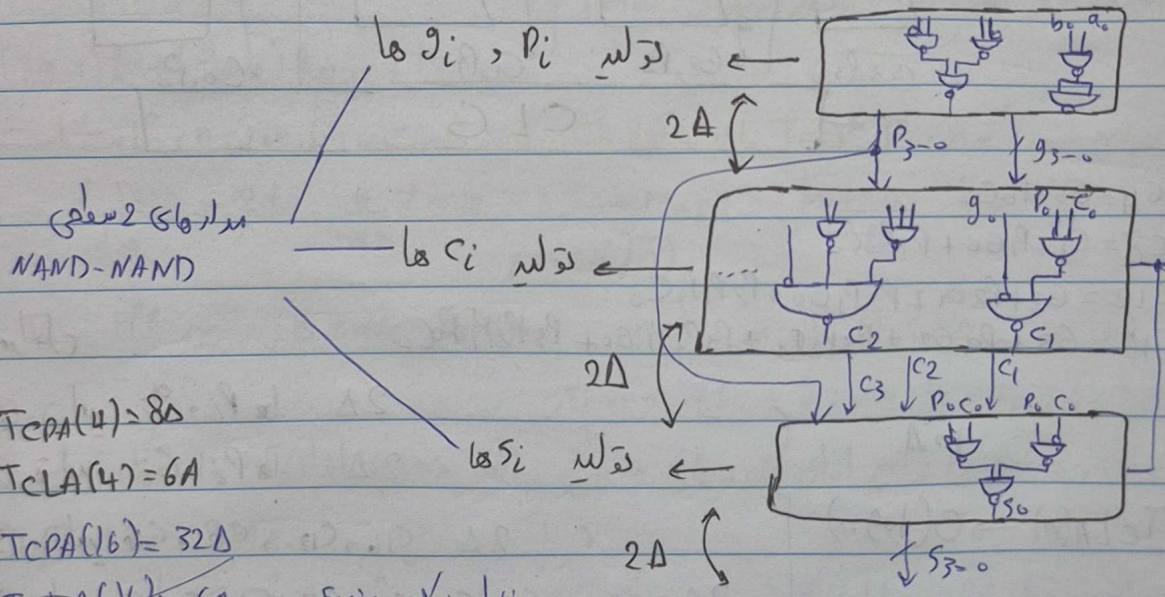
$S_i = a_i \oplus b_i \oplus c_i = P_i \oplus c_i$

$i=0 : c_1 = g_0 + P_0 c_0$

$i=1 : c_2 = g_1 + P_1 c_1 = g_1 + P_1 g_0 + P_1 P_0 c_0$

$i=2 : c_3 = g_2 + P_2 c_2 = g_2 + P_2 g_1 + P_2 P_1 g_0 + P_2 P_1 P_0 c_0$

$i=3 : c_4 = g_3 + P_3 c_3 = g_3 + P_3 g_2 + P_3 P_2 g_1 + P_3 P_2 P_1 g_0 + P_3 P_2 P_1 P_0 c_0$



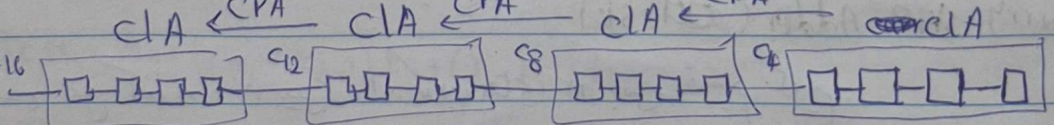
$T_{CPA(4)} = 8\Delta$

$T_{CLA(4)} = 6A$

$T_{CPA(16)} = 32\Delta$

$T_{CLA(16)} = 6\Delta$ ← Fanin مناسب مشکل

برای اینکه مشکل Fanin رو حل کنیم میایم ترکیبی عمل می‌کنیم : مثلا برای 16 بیتی:



CLA_4 تاخیر 12 ، CLA_3 تاخیر 10 ، CLA_2 تاخیر 8 ، CLA_1 برای 6A محاسبه تاخیر

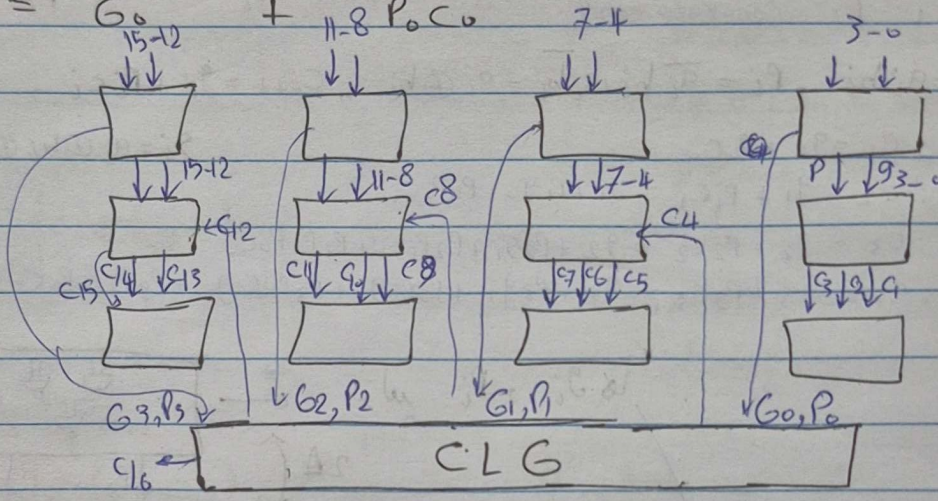
شکل مدار رو توی خونه ببین.

DATE : / / SUB :

میتونیم یه حرکت 200 IQ بزنیم و خودِ این 4 بلوک CLA ها را هم به صورت carry lookahead Adder به هم وصل کنیم که تا حدیش از وصل کردن متوالی کمتره.

$$\begin{cases} C_1 = g_0 + P_0C_0 \\ C_2 = g_1 + P_1g_0 + P_1P_0C_0 \\ C_3 = g_2 + P_2g_1 + P_2P_1g_0 + P_2P_1P_0C_0 \\ C_4 = g_3 + P_3g_2 + P_3P_2g_1 + P_3P_2P_1g_0 + P_3P_2P_1P_0C_0 \end{cases}$$

$$C_4 = G_0 + P_0C_0$$



$$C_4 = G_0 + P_0C_0$$
$$C_8 = G_1 + P_1G_0 + P_1P_0C_0$$
$$C_{12} = G_2 + P_2G_1 + P_2P_1G_0 + P_2P_1P_0C_0$$
$$C_{16} = G_3 + P_3G_2 + P_3P_2G_1 + P_3P_2P_1G_0 + P_3P_2P_1P_0C_0$$

زمان مراحل

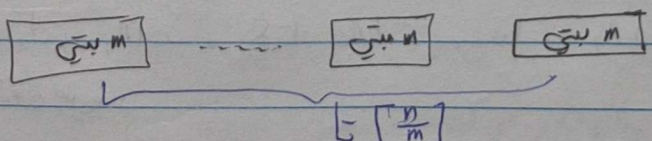
① تولید P_i و g_i ها — 2Δ
② تولید P_i و G_i ها — 2Δ
③ تولید C_{16}, C_{12}, C_8, C_4 — 2Δ
④ تولید $C_1, C_2, C_3, C_5, C_6, C_7, C_9, \dots, C_{15}$ — 2Δ
⑤ تولید S_i ها — 2Δ

10Δ

$T_{CLA}(n) \rightarrow O(\log n)$

Carry skip Adder (CSK)

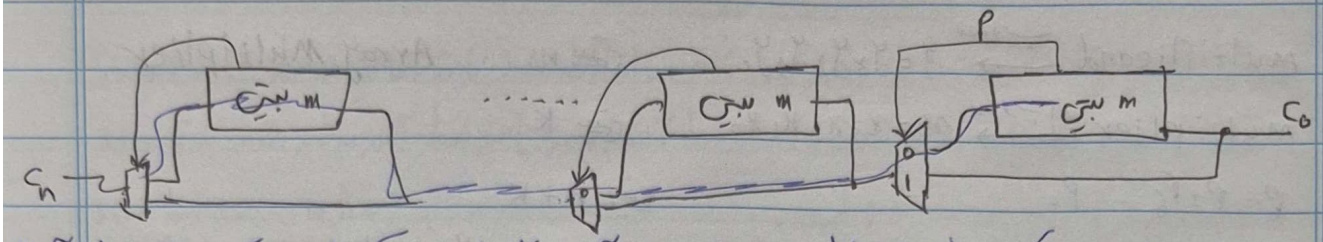
ایده ← جمع کننده n بیتی را در گروه های m بیتی تقسیم کنیم.



★ به جای اینکه carry را به شیوه CPA استفاده کنیم (از یک adder به adder دیگر) carry را سعی می کنیم در مسیر های کوتاه تر استفاده کنیم.

SOBHAN

DATE : / / SUB :



طولانی‌ترین مسیر ممکن برای انتقال carry، مسیر آبی رنگ است که از درون بلوک m بیتی اول، آغاز می‌گردد و بقیه را skip می‌کند.

$$2\Delta = \begin{cases} \star\ t_c \leftarrow \text{تاخیر تولید carry در یک FA} \\ \star\ t_s \leftarrow \text{تاخیر تولید sum در یک FA} \\ \star\ t_{mux} \leftarrow \text{تاخیر یک مالتی پلکسر} \end{cases}$$

$$T_{csk}(n,m) = m t_c + t_{mux} + \left(\frac{n}{m}-2\right) t_{mux} + (m-1) t_c + t_s$$

$m t_c$ ← برای گروه اول
t_{mux} ← برای اولین گروه‌های میانی
$\left(\frac{n}{m}-2\right) t_{mux}$ ← چون کری باید در یکی از FA ها کمین کند در غیر اینصورت از مسیر کوتاهتر عبور می‌کند (گروه آخر)

$$= (2m-1) t_c + \left(\frac{n}{m}-1\right) t_{mux} + t_s$$

برای کمینه کردن $T_{csk}(n,m)$ باید از آن مشتق بگیریم (نسبت به m) چون n، ثابت است.

$$\rightarrow \frac{d\,T_{csk}}{dm} = 2t_c - \frac{n}{m^2} t_{mux} = 0 \rightarrow m_{opt} = \sqrt{n \frac{t_{mux}}{2 t_c}}$$

$\hookrightarrow O(\sqrt{n})$

$$T_{csk}^{opt} \simeq \sqrt{8\, t_{mux} \times t_c \times n}$$

مثال: $n=32, m=4, t_{fa}=2\Delta$

$$T_{csk} = 7t_c + 7t_{mux} + t_s = 15 \times 2\Delta = 30\Delta$$

حالا فرض کنیم می‌توانیم اندازه گروه‌های m بیتی را تغییر دهیم و اندازه آن‌ها به شکل زیر است:

$m_0, m_1, m_2, \dots, m_{k-1}$ (k گروه داریم)

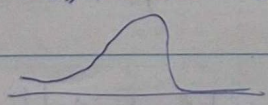
* کری تولید شده در اولین گروه نسبت به گروه‌های میانی، پرش‌های بیشتری دارد

حال فرض کنید کری در گروه i تولید شود و در گروه j خاتمه یابد ($j \geq i$)

← کری باید از $i-1$ تا j گروه میانی عبور کند

$$\begin{cases} T_{csk} = \max\left((m_i + m_j - 1) t_c + (\quad) - i - 1) t_{mux}\right) \\ \sum m_i = n \end{cases}$$

می‌توان نشان داد در این حالت برای کمین T_{csk}^{opt}، اندازه هر گروه به شکل نرمال تغییر می‌کند:



مثال: $n=60, m=60, t_{fa}=2\Delta$

اگر همه m ها برابر باشند $\rightarrow (2m-1) t_c + \left(\frac{n}{m}-1\right) t_{mux} + t_s = 42\Delta$

اگر به صورت نرمال $\rightarrow m_i = 4,5,6,7,8,8,7,6,5,4 \rightarrow T_{csk} = 34\Delta$

SOBHAN

8
CO

DATE : / / SUB :

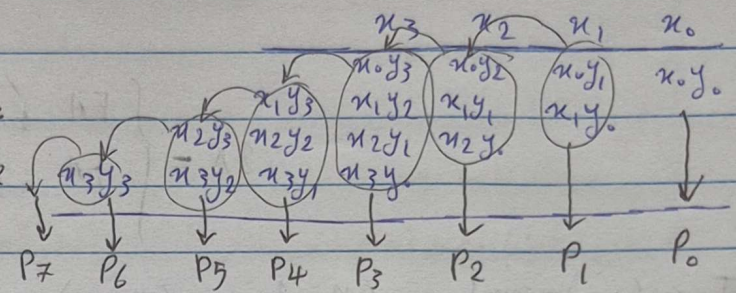
Multiplicand ← مضروب $y = y_3 y_2 y_1 y_0$ n بیتی Array Multiplier

Multiplier ← مضروب الیه $x = x_3 x_2 x_1 x_0$ k بیتی

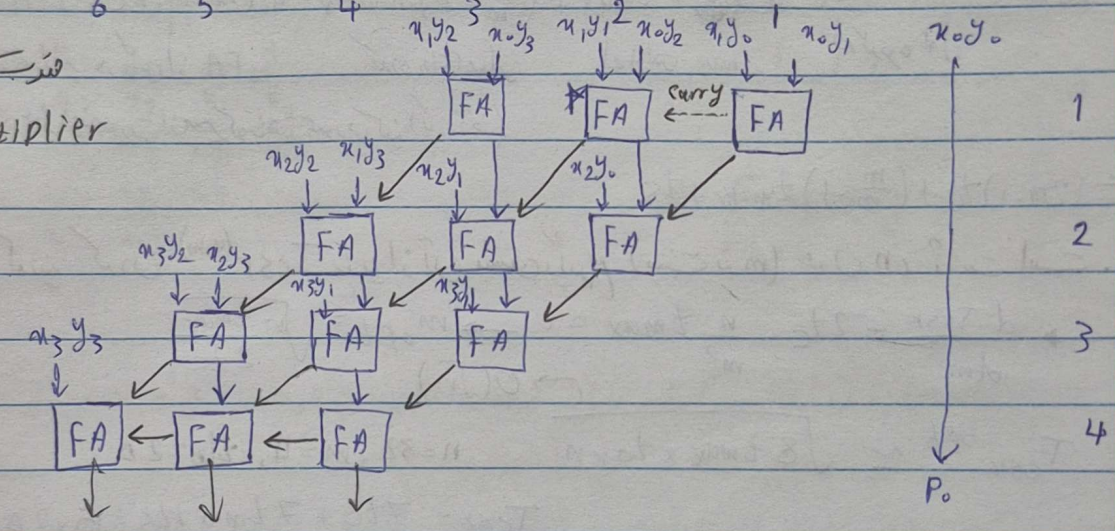
$P = P_7 P_6 \dots P_0$ n+k بیتی

Product حاصلضرب

برای محاسبه سخت افزاری که ضرب دو عبارت را حساب کند، در ابتدا به تعدادی AND gate نیاز داریم که حاصل $x_i y_j$ ها را به دست آورد.



ضرب کننده آرایه ای
array multiplier



FA سطر اول و ستون دوم (*) در صورتی که کری به شکل خطی منتقل شود، باستی منتظر FA سطر اول و ستون اول بماند تا کری را تولید کند. نکته مهم این است که به جای انتقال کری به شکل خطی، آن را به شکل موازی منتقل می کنیم در این صورت FA های سطر اول برای اعلام نتیجه جمع شان نیازی به کری تولید شده توسط FA مقابل خود ندارند و به شکل موازی کار می کنند ← در زمان انجام کار صرفه جویی خواهد شد.

** در سطر آخر، زیر FA ها، FA دیگری نداریم که کری را به شکل موازی انتقال دهیم به همین علت کری را به شکل خطی منتقل می کنیم.

ضرب دو عدد n بیتی :

گیت n^2 : AND برای ساخت $x_i y_j$ ها

n سطر از FA ها داریم با n-1 تا FA ← کلا به $n(n-1)$ FA نیاز داریم.

* در هر سطر به جز سطر آخر carry ها به شکل موازی، منتشر می شوند.

SOBHAN
- حاصل جمع ها، هم به سطر بعدی داده می شوند.
- در سطر آخر کری به شکل خطی، Propagate می شود.

DATE : / / SUB :

محاسبه تاخیر ضرب کننده: اگر تاخیر هر FA برابر با 2Δ باشد، تاخیر هر سطر از FA های Array multiplier برابر 2Δ است. (به جز سطر آخر) و در تاخیر سطر آخر برابر تعداد FA های موجود در سطر آخر ضرب در 2Δ است.

سطر 4 — سطر 3 تا 1

$3\times 2\Delta + 3\times 2\Delta = 12\Delta$

به عنوان مثال تاخیر یک ضرب کننده 4 بیتی برابر است با:

** حتما حتما مثال های ص 16 جزوه (از زبان کارایی) رو بخون ** ص 17 هم مهم نظامی نباز

در Array multiplier با استفاده از سخت افزار زیاد توانستیم به سرعت مطلوب در ضرب کردن برسیم. حال در اینجا می خواهیم سخت افزار را کم تر کنیم (هزینه ارزان تر شود) و در عوض سرعت ضرب مان کم شود.

یک روش این است که تک تک بیت های عدد x را در خود y ضرب کنیم و برای شیفت به چپ اعداد در ضرب در 2 استفاده کنیم.

$$y = y_3\ y_2\ y_1\ y_0$$
$$* \quad x_3\ x_2\ x_1\ x_0$$

$x_0y_3\ x_0y_2\ x_0y_1\ x_0y_0 \longrightarrow x_0 2^0 y$

$x_1y_3\ x_1y_2\ x_1y_1\ x_1y_0 \longrightarrow x_0 2^1 y$

$x_2y_3\ x_2y_2\ x_2y_1\ x_2y_0 \longrightarrow x_0 2^2 y$

$x_3y_3\ x_3x_2\ x_3y_1\ x_3y_0 \longrightarrow x_0 2^3 y$

$$\rightarrow P = \sum_{i=0}^{3} x_i 2^i y$$

ایراد این روش این است که باید تا پایان عملیات جمع، $x_0 2^0 y$ ، $x_1 2^1 y$ ، $x_2 2^2 y$ ، $x_3 2^3 y$ را داشته باشیم تا بتوانیم عملیات جمع را انجام دهیم.

برای رفع این ایراد از حاصل ضرب انباشته شده استفاده می کنیم به این ترتیب که در ابتدا متغیری مانند P تعریف می کنیم که مقدار اولیه اش صفر است. سپس در هر مرحله به P، مقدار $x_i 2^i y$ را اضافه می کنیم. ($P = P + x_i 2^i y$) Partial Product

برای انجام ضرب با استفاده از روش حاصلضرب انباشته، روابط زیر برقرار اند:

$P_0 = 0$

$P_{i+1} = P_i + x_i 2^i y$

مثال →

$$\begin{array}{r} 1010 \\ 1101 \times \end{array}$$

0000 0000 $P_0 = 0$

+ 1010 $\rightarrow x_0 y$

0000 1010 $P_1 = P_0 + x_0 2^0 y$

+ 0000 $\rightarrow x_1 y$

0000 1010 $P_2 = P_1 + x_1 2^1 y$

+ 1010 $\rightarrow x_2 y$

0011 0010 $P_3 = P_2 + x_2 2^2 y$

+ 1010

1000 0010 $P_4 = P_3 + x_3 2^3 y$

SOBHAN

DATE : / / SUB :

ایراد دیگری که این روش دارد این است که بجای y_i، هر بار بسته به متفاوتی از حاصلضرب جمع می شود.

برای رفع این ایراد، بیت های مشخص شده در صفحه قبل، به محض محاسبه کردن تا پایان عملیات ثابت اند. همچنین در هر مرحله این y_i است که یک واحد به سمت چپ شیفت می خورد می توانیم به جای شیفت دادن y_i، خود P_i را شیفت دهیم ولی P_i را باید به سمت راست شیفت داد ← خواهیم داشت: (y_i همواره با بیت سمت چپ P_i جمع خواهد شد)

$P_0 = 0$, $P_i = P_i + x_i y$ ← دیگر 2^i نداریم چون y_i را شیفت نمی دهیم

$P_{i+1} = P_i \times 2^{-1}$ ← در عوض P_{i+1} شیفت به راست می خورد

مثال: 1010×1101

$P_0 \leftarrow 0000\ 0000$

$x_0 y \leftarrow 1010$

کری ← 0 1010 0000

$P_1 \leftarrow 0101\ 0000$

$x_1 y \leftarrow 0000\ 0000$ ← یک واحد shift به راست

کری ← 0 0101 0000

$P_2 \leftarrow 0010\ 1000$ ← یک واحد شیفت به راست

1010

کری ← 0 1100 1000

$P_3 \leftarrow 0110\ 0100$ ← یک واحد شیفت به راست

1010

کری ← 1 0000 0100

$P_4 \leftarrow 1000\ 0010$ ← یک واحد شیفت به راست

Add & shift-L multiPlier

در مقایسه با ضرب آرایه ای فقط از یک adder استفاده می کند و در عوض کندتر است.

Division

در این درس فقط به تقسیم اعداد علامت دار مثبت پرداخته می شود (بعد جبر هر دو عددی که مقدار است به هم تقسیم کنیم، با بیتی داشته باشیم)

معکوس (Dividend) 2n بیت	معکوس علیه (Divisor) n بیت
باقی مانده n بیت	خارج قسمت n بیت

محدودیت ها:

① تقسیم بر 0: معکوس علیه = 0 → Exception

Divide by 0

② سرریز (over flow):

مثال → 0100 \| 01 / 100 ← سرریز

اگر رابطه زیر برقرار باشد، سرریز رخ نخواهد داد.

n بیت بیشتر از ارزش معکوس علیه > n بیت بیشتر از ارزش معکوس.

SOBHAN همانطور که دیدید راهی نداشتیم با جمع های متوالی انجام دهیم، تقسیم هم با تفریق های متوالی قابل انجام است.

ص ۱۱

DATE : / / SUB :

الگوریتم تقسیم (روش Restoring):

A, Q ← مقسوم (مقسوم را در رجیستر های A و Q می ریزیم)

① shift AQ (A و Q را به چپ شیفت می دهیم)

② sub dir (A - مقسوم)

③ Add dir اگر حاصل منفی است:

set $Q_0 = 1$ در غیر اینصورت:

مراحل فوق تقریبا یک حلقه for، n بار تکرار می شوند

مثال: تقسیم عدد 75+ بر عدد 11+

A: 00010 Q: 10011 → 2 شیفت → 10101

روش Non-Restoring: در این روش، زمانی که باقیمانده منفی شود، تقسیم را ادامه می دهیم و در مرحله بعدی تقسیم به جای انجام عملیات تفریق، عملیات جمع انجام می دهیم (به جای جمع زوج با مکمل 2 مقسوم علیه، با خود مقسوم علیه جمع می زنیم)

روش Restoring:

	A	Q
	0 0000	01011
① shift چپ	00010	10110
sub	11101	
منفی!	11111	
Add dir	00101	
	00010	
② shift چپ	00100	01100
sub	11101	
منفی!	11111	
Add dir	00101	
	00100	
③ shift چپ	01000	11000
sub	11101	
مثبت $Q_0 \leftarrow 1$	00101	11001
④ shift چپ	01011	10010
sub	11101	
مثبت $Q_0 \leftarrow 1$	01000	10011
⑤ shift چپ	10001	00110 → خارج قسمت +6
sub	11101	
منفی!	11110	
Add dir	00101	
	10001	← باقیمانده +5

SOBHAN

DATE : / / SUB :

DataPath, Controler

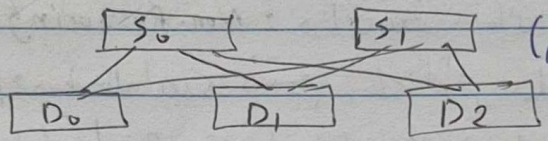
سیستم دیجیتال سنکرون : سیستمی است که در آن تمام المان های حافظه به یک لبه CLK حساس هستند (به طور موازی کار میکنند) یعنی تمام المان های حافظه یا با لبه بالارونده CLK کار میکنند یا لبه پایین رونده.

★ سیستم دیجیتال سنکرون از دو بخش تشکیل شده است ① مسیر داده ② واحد کنترل

مسیر داده از اتصال بسیاری واحد محاسباتی (+ و - و * و ...) و تعدادی المان حافظه (FF و Reg و SReg و ...) تشکیل شده است که توسط گذرگاه به یکدیگر متصلند.

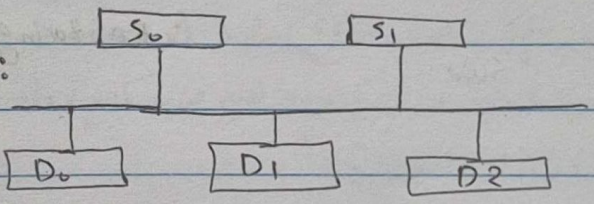
★ گذرگاه (Bus) : مجموعه ای از سیم هاست که برای انتقال داده از یک نقطه به نقطه ای دیگر استفاده می شود. انواع Bus ها: ① Dedicated (اختصاصی) ② Common (اشتراکی)

Dedicated :



(بین هر زوج مبدا و مقصد یک Bus اختصاصی داریم)
مزایا : امکان انتقال همزمان داریم.
ایراد : پرهزینه.

Common :



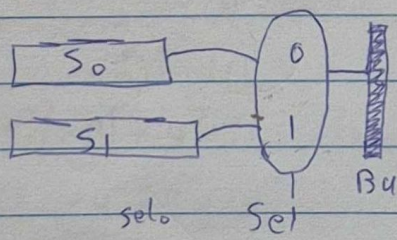
از یک گذرگاه اشتراکی استفاده می کنیم
- انتقال همزمان نداریم !
+ کم هزینه است.

★ در طول درس از Bus اشتراکی استفاده می کنیم.

نکته حائز اهمیت در Bus اشتراکی این است که دو فرستنده همزمان روی Bus، اطلاعات نفرستند.

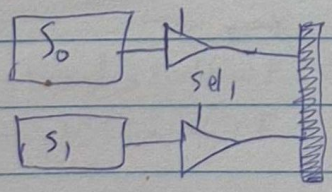
نحوه پیاده سازی Bus اشتراکی :

① استفاده از مالتی پلکسر : در این حالت هواره Bus اشغال است (یا توسط S0 یا S1)

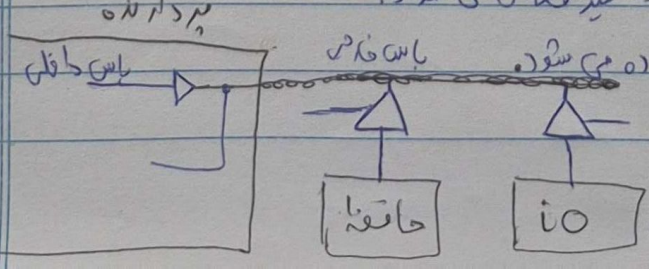


عموماً برای Bus های داخلی استفاده می شود.

② استفاده از tri-state :



باید تضمین دهیم که Sel0 و Sel1 همزمان باهم فعال نشند.
در این حالت به خاطر استفاده از tri-state اگر Sel0 و نه Sel1 فعال باشند، Bus غیرفعال می شود.



عموماً برای Bus های خارجی استفاده می شود.

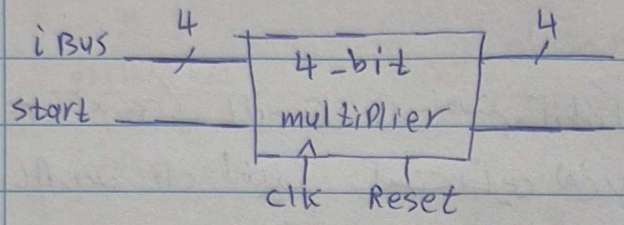
به عنوان مثال اگر در شکل زیر بخواهیم روی باس خارجی، مقداری قرار دهیم، باید هر دو حافظه و IO غیرفعال باشند. (اگر از مالتی پلکسر استفاده شده مشکل چندین آی او ایجاد نمی شد.)

SOBHAN

DATE : / / SUB :

تعریف مجدد مسیر داده: تمامی مسیرهای لازم برای انجام عملیات مشخص در سیستم را فراهم می کند.

مثال: طراحی مسیر داده برای multiplier Add & shift:



میخواهیم دو عدد 4 بیتی را در هم ضرب شان را پیدا کنیم

- در 2 مرحله اعداد 4 بیتی را از طریق iBus به ضرب کننده می دهیم.
- حاصل ضرب هم 8 بیتی می شود، و OBus 4 بیتی است ← حاصلضرب را هم در دو مرحله (دو clk) نمایش می دهیم.
- سیگنال start نشان میدهد که چه زمانی عملیات ضرب شروع شده است.
- سیگنال Done نشان میدهد چه زمانی عملیات ضرب به اتمام رسیده
- سیگنال start با لبه بالارونده clk کار می کند.

در اولین لبه بالارونده clk عملیات ضرب شروع میشود (start هم باید 1 باشد.)

- در دومین و سومین لبه بالارونده، دو عددی که می خواهیم ضرب کنیم، وارد می شوند.
- در دومین لبه بالارونده ابتدایی، حاصل نمایش داده می شود.

Registers 4bit

X = Multiplier;

Y = Multiplicand;

Register 8bit P[7:0] = 0;

E = 0; برای ذخیره کردن carry در عملیات جمع

```
for(i=0; i<4; i++){
    {E,P[7:4]} = P[7:4] + X_i × Y;
    P = P >> 1;   // شیفت P به راست
}
```

حاصل جمع، و carry در رجیترهای P[7:4] و E ذخیره می شوند.

* اگر $x_i = 0$ باشد، هیچ عملیاتی رخ نمیدهد ولی اگر $x_i = 1$ باشد، آنگاه:

```
{E,P[7:4]} = P[7:4] + Y;
```

```
for(i=0; i<4; i++){
    if(x_i == 1)
        {E,P[7:4]} = P[7:4] + Y;
    P = P >> 1;
}
```

load ← رجیتر 4 بیتی با قابلیت ← X

load ← رجیتر 4 بیتی با قابلیت ← Y

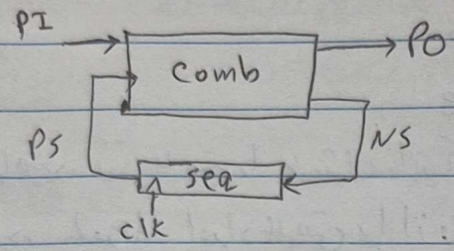
clear, load, shift : P

load, clear با قابلیت FF یک : E

*** شکل مدار در صفحه 25 و 26 جزوه کارایی

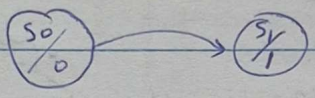
DATE : / / SUB :

روش هاسمن : هر مدار ترتیبی از دو بخش combinational و sequential تشکیل شده است.
sequential از تعدادی gate تشکیل شده است. و یا از تعدادی FF تشکیل شده است

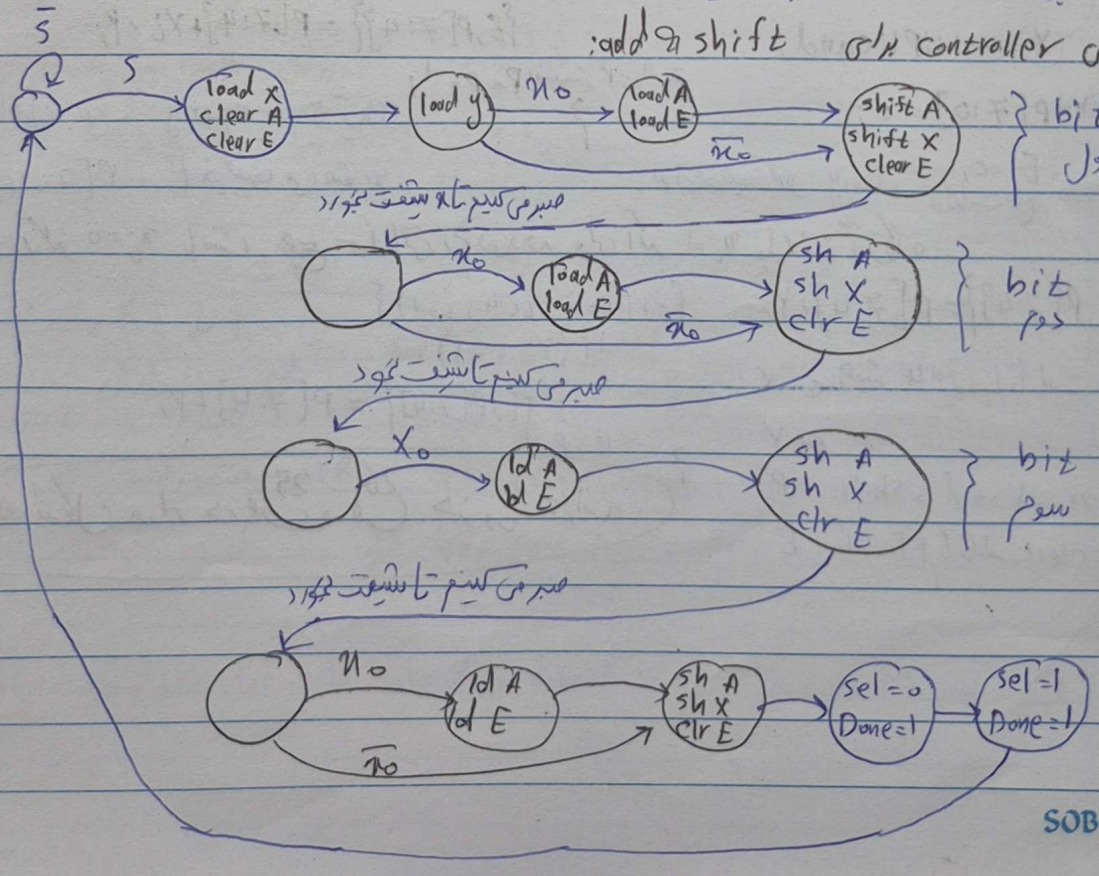


★ مدارهایی که روی یک بازه زمانی در حال حرکت هستیم و تا زمانی که لبه بالارونده clk را نبینیم، به استیت نهایی نمی رویم.
مدارهای mealy به کارمون نمیاد و فقط moore داریم.
مدارهایی که Ns، تابعی از Ps و PI است ولی Po، فقط تابعی از Ps است:

$$\begin{cases} N_S = F(P_S, P_I) \\ P_O = G(P_S) \end{cases}$$



زمانی که وارد یک استیت میشویم و مقادیر صادر می کنیم، فرمان صادر شده، بدون تاخیر انجام نمی شود و با مقداری تاخیر انجام خواهد شد.
زمانی که یک فرمان مانند shift، load و... در حال انجام شدن است، نمی توان به شکل همزمان مقدار جدید متغیر را بررسی کرد.
وظیفه controller این است که سیگنال های کنترلی مورد نیاز data Path را تولید کند.
در مقابل، data Path هم وضعیت خود را به controller اعلام می کند.
مراحل controller برای add & shift:



SOBHAN

DATE : / / SUB :

Booth Multiplier ← برای ضرب اعداد علامتدار

* یادآوری: در ضرب کننده add & shift، تعداد جمع زدن ها با تعداد 1های multiplier و تعداد شیفت ها با تعداد بیت های multiplier برابر است.

ضرب دستی Booth.

① باید multiplier را بروش Booth کد کنیم (هر دو بیت متوالی را نگاه می کنیم)

x_i	x_{i-1}	operation
0	0	0 → No operation
0	1	1 → sum
1	0	1 → sub
1	1	0

نکته: برای کد کردن باید یک صفر به سمت راست اضافه کنیم.

مثال → 00111 ⟵ 00111

② $1 * y = -y$ (مکمل 2)، $1 * y = y$ ، $0 * y = 0$

③ عمل جمع را انجام می دهیم ← قبل از انجام عمل جمع، عدد آخر را به اندازه یک بیت sign extend می کنیم و بقیه اعداد را به اندازه 0 ای sign extend می کنیم که سمت چپ آن با عدد آخر تراز شود.

مثال → $y = 10 = 01010$ ، $x = -13 = 10011$

x کد شده → 01101 ، $\bar{y} = 10110$ ، $y \rightarrow 01010$

$\bar{y}$ 01101

01010

11111 10110

00000

00101

+ 11010

* 1101111110 = -130

اگر عددی داشته باشیم که تعداد 0 متوالی و تعداد 1 متوالی داشته باشد روش Booth بسیار بهینه است ولی در حالتی که بیت ها از 0 و 1 یا از 1 و 0 تشکیل شده باشد این روش بهینه نیست.

نکات:

① به E و FF برای ذخیره کردن عمل جمع نیازی نداریم

② در شیفت به راست A و X از سمت چپ A، بیت علامت A وارد می شود

شبه کد:

```
X-1 = 0;   همان صفری که سمت راست عدد اضافه می کنیم
A = 0;
for(i=0; i<n; i++){
  if(xi x(i-1) == 01)
    A = A + y;
  elif(xi x(i-1) == 10)
    A = A - y;
  shr {A, X, x-1}
```

③ به یک FF نیاز داریم که ابتدای کار مقدار x_{-1} را نگه دارد. (0) و بعد از شیفت به راست، مقادیر را ذخیره کند.

مثال

DATE : / / SUB :

Floating Point

$\bar{a}.\bar{N} * 10^{2}$ (نماد علمی)

(E)
(m) Exponent
$(-1)^{S} \text{Mantissa} * 2 = N$

در مبنای دو ←

$S = \begin{cases} 0 \rightarrow \text{عدد مثبت} \\ 1 \rightarrow \text{عدد منفی} \end{cases}$

هر چقدر تعداد بیت های مانتیس بیشتر باشد، دقت نمایش هم بیشتر می شود.
هر چقدر تعداد بیت های توان، بیشتر باشد، محدوده نمایش بیشتر می شود.

دو نوع داریم : (IEEE-754)

① میز شناور با دقت ساده (SPFP) :

1	8	23
S	E	M

② میز شناور با دقت مضاعف (DPFP) :

1	11	52
S	E	M

حالات خاص : ۰ ← همه بیت ها ۰
$\pm\infty$ ← با توجه به sign همه بیت ها ۱

NaN $\frac{0}{0}$, $\sqrt{-1}$

مثال : $N = -13.125 * 2^{0}$

$N = -13,125 \times 2^{0} = -1101.001 \times 2^{0}$

$= -1.101001 \times 2^{3}$ ⑤ → $E = 127 + 3 = 130$

S ← ، مانتیس

S	E	M
1	10000010	10100100000000000000000

23 بیت

BCD Adders (Binary coded Decimal)

مثال → 137 → 0001 0011 0111

حالت ①: از جمع دو رقم مجاز BCD یک رقم مجاز BCD بدون carry به دست آمده است.

$\begin{array}{r} 2 \\ +\ 3 \\ \hline 5 \end{array}$ $\begin{array}{r} 0010 \\ +0011 \\ \hline 0101 \end{array}$

← در این حالت باجمع کننده Binary هم می توانیم کار کنیم.

حالت ②: از جمع دو رقم مجاز BCD یک رقم غیر مجاز BCD بدون carry به دست آمده است.
در این حالت از عدد باینری به دست آمده باید ۱۰ را کم کنیم یا با مکمل ۲ آن را با عدد جمع کنیم و یک واحد هم کری به بیت بعدی بدهیم.

$\begin{array}{r} 5 \\ +\ 7 \\ \hline 12 \end{array}$ $\begin{array}{r} 0101 \\ +0111 \\ \hline 1100 \end{array}$ $\begin{array}{r} 1100 \\ +0110 \\ \hline 10010 \end{array}$

حالت ③: از جمع دو رقم مجاز BCD، یک رقم مجاز BCD همراه با carry تولید شده است.
مشابه با حالت قبل، ۱۰ را کم می کنیم و carry را بیت بعدی می دهیم.

$\begin{array}{r} 9 \\ +\ 9 \\ \hline 18 \end{array}$ $\begin{array}{r} 1001 \\ 1001 \\ \hline ①0010 \end{array}$

حال برای مقادیر K, Z_8, Z_4, Z_2, Z_1 می خواهیم بفهمیم چه زمانی به تصحیح نیاز داریم:

$K\ Z_8\ Z_4\ Z_2\ Z_1$	f	Cout	$S_3 S_2 S_1 S_0$	
0 0 0 0 0	0	0	0 0 0 0	حالت ①
0 0 0 0 1	0	0	0 0 0 1	نیاز به تصحیح نداریم!
⋮	⋮	⋮	⋮	
0 1 0 0 1	0	0	1 0 0 1	
0 1 0 1 0	1	0	0 0 0 0	حالت ②
0 1 0 1 1	1	0	0 0 0 1	نیاز به تصحیح داریم
⋮	⋮	⋮	⋮	
0 1 1 1 1	1	0	1 0 0 1	
1 0 0 0 0	1	0	0 0 0 0	حالت ③
1 0 0 0 1	1	0	0 0 0 1	نیاز به تصحیح داریم
⋮	⋮	⋮	⋮	
1 0 0 1 1	1	1	0 0 1 1	

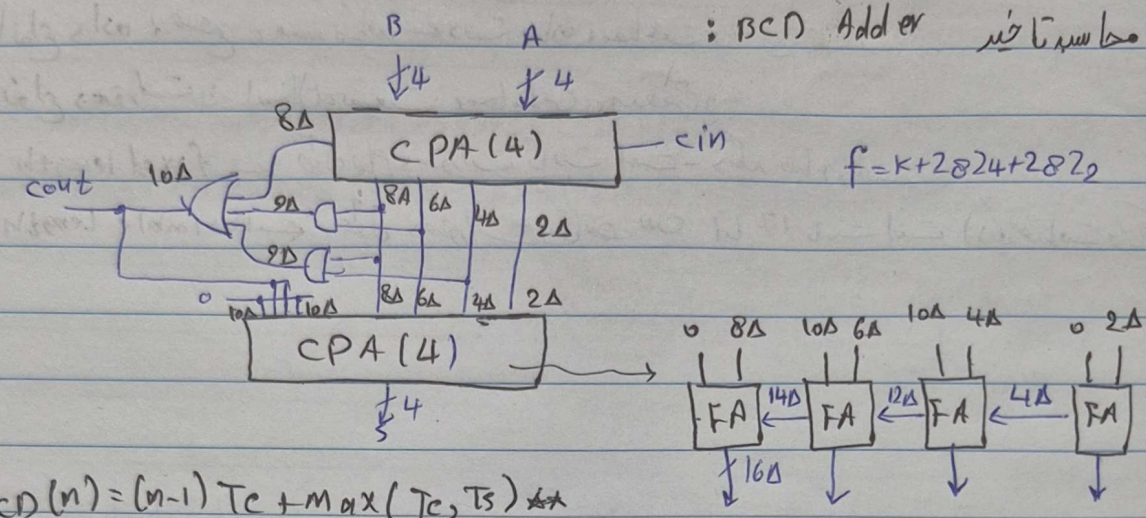
→ $f = K + Z_8 Z_4 + Z_8 Z_2$

$F = Cout$

SOBHAN

DATE : / / SUB :

محاسبه تاخیر BCD Adder :



$f = k + 2824 + 282_2$

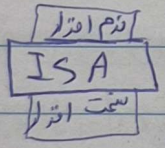
$T_{BCD}(n) = (n-1)T_c + \max(T_c, T_s)$

* اگر دکوری برای محاسبه مکمل ۹ یک عدد کافی باشد که مکمل اعداش آن را محاسبه کنیم، به این کد، خود مکمل گفته می‌شود.

* اگر اعداش باید برای اعداد BCD را با جمع بزنیم، به یک کد خود مکمل می‌رسیم. از این اعداد در ساخت تفریق کننده BCD استفاده می‌شود.

(ISA) : Instruction Set Architecture

ISA : یک واسط ارتباطی بین سخت افزار و نرم افزار است که اطلاعات مورد نیاز سخت افزار را برای نرم افزار فراهم می‌کند. همچنین اطلاعات مورد نیاز نرم افزار را برای سخت افزار فراهم می‌کند.



① مسیر داده پردازنده:

① طراحی معماری مجموعه دستورات پردازنده ① مسیر داده ② واحد کنترل

② کلاس پردازنده:

* کلاس Reg-mem ← آپرند دستورات محاسباتی می‌تواند محتویات یک خانه حافظه باشد.

* کلاس load/store: load ← خواندن از حافظه، store ← نوشتن در حافظه — در درس با این کلاس کار می‌کنیم.

پردازنده MIPS از این نوع است.

② سازماندهی حافظه: * دسترسی به حافظه چگونه است؟ * دسترسی aligned سریع‌تر است

* unaligned یا aligned بودن حافظه

در پردازنده MIPS، آدرس‌ها همواره aligned هستند.

MIPS ← Little Endian ← دیتای رشته باید در آدرس کوچکتر، دیتای رشته بالادر آدرس بزرگتر

Big ← دیتای رشته باید در آدرس بزرگتر و دیتای رشته بالادر آدرس کوچکتر

SOBHAN